

CS1570 Lecture Notes

Claire Mathieu

Fall 2026

1 September 29 and October 1: Edit distance

1.1 Edit distance and alignment cost

Let Σ be a finite alphabet and let $- \notin \Sigma$ be a special *gap* symbol. For a string $x = x_1x_2 \cdots x_n \in \Sigma^*$ we write $|x| = n$, and $x[1..i] = x_1 \cdots x_i$ denotes its prefix of length i (with $x[1..0] = \varepsilon$, the empty string).

Definition 1 (Edit distance). An *edit operation* on a string is one of the following: **insertion** of a character of Σ at an arbitrary position; **deletion** of one character; **substitution** of one character by a different character of Σ . An *edit script* from x to y is a finite sequence of edit operations which, applied successively starting from x , produces y . The *edit distance* $\text{ed}(x, y)$ is the minimum length of an edit script from x to y .

For example, **MONEY** vs. **FOOD**. One optimal sequence is

MONEY $\rightarrow$ MONE $\rightarrow$ FONE $\rightarrow$ FOOE $\rightarrow$ FOOD,

so the distance is 4. Distance 3 is not possible: the letters M, N, E, Y each need to be deleted or changed. Other examples: **BOX** $\rightarrow$ **BORING** takes insertions/substitutions (**BOX** $\rightarrow$ **BOR** $\rightarrow$ **BORI** $\rightarrow$ **BORIN** $\rightarrow$ **BORING**), and a pair such as **ALGORITHMS** / **DYNAMIC** can be visualized by lining up the strings and marking which positions agree.

Definition 2 (Alignment and alignment problem). An *alignment* of x and y is a sequence of T *columns*

$$\begin{pmatrix} u_1 \\ v_1 \end{pmatrix}, \begin{pmatrix} u_2 \\ v_2 \end{pmatrix}, \dots, \begin{pmatrix} u_T \\ v_T \end{pmatrix}, \quad (u_t, v_t) \in (\Sigma \cup \{-\})^2 \setminus \{(-, -)\},$$

such that deleting the gap symbols from $u = u_1 \cdots u_T$ yields x , and deleting the gap symbols from $v = v_1 \cdots v_T$ yields y . Each column is of one of three types: a *match/mismatch* (a, b) with $a, b \in \Sigma$, a *deletion* $(a, -)$, or an *insertion* $(-, b)$. The cost $c(u_t, v_t)$ of column t is 0 if $u_t = v_t$ and 1 otherwise, and the cost of an alignment is the sum of the costs of its columns. The *alignment problem* asks for the minimum cost $\text{al}(x, y)$ over all alignments of x and y .

Lemma 1. For all strings $x, y \in \Sigma^*$, $\text{ed}(x, y) = \text{al}(x, y)$.

Proof. To prove an equality, the standard organisation of a proof is to prove the two directions $\leq$ and $\geq$. We first prove the easy direction, namely, that $\text{ed}(x, y) \leq \text{al}(x, y)$.

Let $A = (u_t, v_t)_{1 \leq t \leq T}$ be an alignment of x and y of cost c . We simply build an edit script of length exactly c by processing the columns from left to right, associating to each column an operation in a straightforward manner:

- (a, b) with $a = b$: no operation (cost 0);
- (a, b) with $a \neq b$: substitute the corresponding character a by b (cost 1);
- $(a, -)$: delete the corresponding character a (cost 1);
- $(-, b)$: insert b at the corresponding position (cost 1).

To prove the other direction, $\text{ed}(x, y) \geq \text{al}(x, y)$, assume that an edit script of length k transforms x into y . We will argue that there is an alignment of x and y of cost at most k . This is done by induction on k .

Base case $k = 0$. Then $x = y$, and the natural alignment has cost 0.

Inductive step $k \geq 1$. Let o_1 be the first operation of the script and let x' be the string obtained from x after applying o_1 . The remaining $k - 1$ operations transform x' into y , so by the induction hypothesis there is an alignment $A' = (u'_t, v'_t)_t$ of x' and y of cost at most $k - 1$. We modify A' into an alignment A of x and y whose cost exceeds that of A' by at most 1. Three cases arise.

- o_1 is a **substitution** of the p -th character x_p by a character $c \neq x_p$. Then $x'_p = c$, and the p -th non-gap symbol of u' is c . Replace this symbol by x_p . The top row now yields x after removing gaps, and only one column changes. Its cost was in $\{0, 1\}$ and is still in $\{0, 1\}$, so the total cost increases by at most 1.
- o_1 is a **deletion** of x_p . Then $x' = x_1 \cdots x_{p-1} x_{p+1} \cdots x_n$. Insert the new column $(x_p, -)$ into A' immediately after the column holding the $(p - 1)$ -th non-gap symbol of u' (at the very beginning if $p = 1$). The top row now yields x , the bottom row still yields y , and the cost increases by exactly 1.
- o_1 is an **insertion** of a character c so that $x'_p = c$ and x is x' with this character removed. The p -th non-gap symbol of u' is c , lying in some column (c, v) .
 - If $v = -$, delete this column from A' . The result is still an alignment (no column is $(-, -)$) and its cost decreases by 1.
 - If $v \in \Sigma$, replace the column by $(-, v)$. Its cost becomes 1, whereas it was ≥ 0 , so the total cost increases by at most 1.

In every case $\text{cost}(A) \leq \text{cost}(A') + 1 \leq k$. □

1.2 The dynamic program

To solve the edit distance problem with a dynamic program, the first step is to find a recurrence relation. Since we have equivalence between edit distance and alignment, let us do the recurrence for the alignment problem, which is more structured and therefore simpler to think about. Let $x = x_1 \cdots x_n$ and $y = y_1 \cdots y_m$. For $0 \leq i \leq n$ and $0 \leq j \leq m$, define

$$D(i, j) = \text{al}(x[1..i], y[1..j]).$$

Theorem 1. $D(0, 0) = 0$, for all i we have $D(i, 0) = i$ and for all j we have $D(0, j) = j$; moreover, for $i, j \geq 1$:

$$\begin{aligned} D(i, j) &= D(i - 1, j - 1) \text{ if } x_i = y_j \\ D(i, j) &= \min \left\{ D(i - 1, j) + 1, D(i, j - 1) + 1, D(i - 1, j - 1) + 1 \right\} \text{ otherwise,} \end{aligned}$$

Proof. **Base cases.** easy by definition of alignment.

General case $i, j \geq 1$. Let A be any alignment of $x[1..i]$ and $y[1..j]$, and consider its last column.

Assume that $x_i = y_j$. The last column of the alignment might as well have x_i aligned with y_j since that costs 0. Indeed, in any alignment, anything else could be transformed into that without increasing the cost: imagine that the last column has x_i aligned with $-$, then scan the alignment right to left until you find y_j on the bottom row, and on the bottom row swap that y_j with the final $-$.

Assume that $x_i \neq y_j$. The last column is not $(-, -)$, so it is one of:

- (a) $(x_i, -)$, of cost 1;
- (b) $(-, y_j)$, of cost 1;
- (c) (x_i, y_j) , of cost 1.

That gives the recurrence. □

To design a dynamic program, we need an array. The array is suggested by the recurrence: array entry $D[i, j]$ for $D(i, j)$, with i going from 0 to m and j from 0 to n .

Finally we need an order in which to fill the array. Since the recurrence requires the entries that are in the previous row or the same row and previous column, we can simply fill the array row by row in the natural order.

Define the table

$$D[i, j] = \text{dist}(A[1..i], B[1..j]), \quad 0 \leq i \leq m, 0 \leq j \leq n,$$

of size $O(mn)$. Each entry $D[i, j]$ depends only on $D[i - 1, j - 1]$, $D[i - 1, j]$, and $D[i, j - 1]$.

Order to fill the table. Any order in which these three neighbours are filled first works, e.g. row by row. When filling row i , only the *previous row* and the *current row* are ever needed; rows above that are not needed anymore.

Algorithm 1 Edit distance DP

```

1: for  $j = 0$  to  $n$  do  $D[0, j] \leftarrow j$ 
2: end for
3: for  $i = 1$  to  $m$  do
4:    $D[i, 0] \leftarrow i$ 
5:   for  $j = 1$  to  $n$  do
6:     if  $A[i] = B[j]$  then
7:        $D[i, j] \leftarrow D[i - 1, j - 1]$ 
8:     else
9:        $D[i, j] \leftarrow 1 + \min(D[i - 1, j - 1], D[i - 1, j], D[i, j - 1])$ 
10:    end if
11:  end for
12: end for
```

Runtime: $O(mn)$.

Space: $O(mn)$.

1.3 Reducing Space Usage

Can you reduce the space to linear, $O(m + n)$? **Answer:** Yes. Only remember the current row and the previous row. This means we can reuse the space of old rows, so we only need an array with two rows, filled by alternating.

1.4 What if we want the sequence of transformations?

What if we want not just the edit distance but also the actual *sequence* of transformations? **Answer:** use an auxiliary table (recording which of the three options achieved the minimum in each cell, then trace back). This brings back $O(mn)$ space.

Summary of what we know:

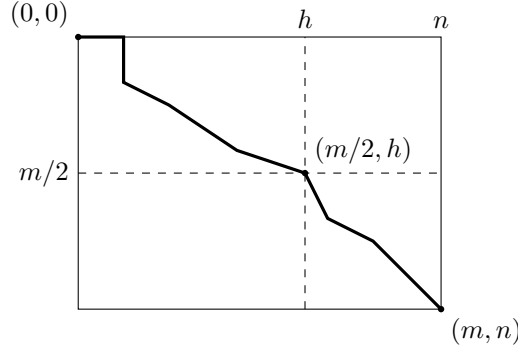
	time	space
DP for edit distance	$O(mn)$	$O(m + n)$
DP for edit sequence	$O(mn)$	$O(mn)$

Goal for Thursday: compute the edit <i>sequence</i> in time $O(mn)$ and space $O(m + n)$.

2 Computing the Edit Sequence in Linear Space

2.1 Idea: divide and conquer

Picture the DP table with rows indexed by $i \in [0, m]$ (positions in A) and columns by $j \in [0, n]$ (positions in B). An optimal edit sequence is a monotone path from $(0, 0)$ to (m, n) in this grid. The path must cross the middle row $i = m/2$ at some column h (“for some h ”).



The path splits into two pieces, giving two independent subproblems:

$$\text{dist}(A[1..m], B[1..n]) = \text{dist}(A[1..\frac{m}{2}], B[1..h]) + \text{dist}(A[\frac{m}{2} + 1..m], B[h + 1..n]).$$

So if we can compute h , we recurse on the top-left rectangle of size $\frac{m}{2} \times h$ and the bottom-right rectangle of size $\frac{m}{2} \times (n - h)$. The recursion keeps cutting rectangles in half (in the A direction), as in the picture of nested rectangles from lecture.

2.2 Computing h with a DP

Definition 3. Let $h(i, j)$ be such that the optimal sequence from $A[1..i]$ to $B[1..j]$ goes through the cell $(\frac{m}{2}, h(i, j))$. The one we are interested in is $h = h(m, n)$.

We compute $h(i, j)$ with a DP alongside $D[i, j]$:

- If $i < \frac{m}{2}$: $h(i, j) = \text{undefined}$ (the path has not reached the middle row yet).
- If $i = \frac{m}{2}$: $h(i, j) = j$ (the path is in the middle row at column j).
- If $i > \frac{m}{2}$: $h(i, j)$ depends on the recurrence for $d(i, j)$: it is inherited from whichever neighbour $((i - 1, j - 1), (i - 1, j), \text{ or } (i, j - 1))$ achieved the minimum (or from $(i - 1, j - 1)$ when $A[i] = B[j]$).

Since each row of h only needs the previous row, $h(i, j)$ can be computed together with $D(i, j)$ in the same DP, as a table of auxiliary variables, in time $O(mn)$ and space $O(m + n)$.

Algorithm 2 Divide-and-conquer edit sequence (sketch)

```

1: procedure EDITPATH( $A[1..m], B[1..n]$ )
2:   if  $m$  or  $n$  is small then
3:     solve directly with the standard DP return
4:   end if
5:   Compute  $D$  and  $h(\cdot, \cdot)$  with two rolling rows; let  $h \leftarrow h(m, n)$ 
6:   EDITPATH( $A[1..\frac{m}{2}], B[1..h]$ )
7:   EDITPATH( $A[\frac{m}{2} + 1..m], B[h + 1..n]$ )
8:   concatenate the results to get the global output
9: end procedure

```

2.3 Analysis

2.3.1 Time

Let $T(m, n)$ be the runtime to compute the path from $A[1..m]$ to $B[1..n]$. Then (plus the base case)

$$T(m, n) = cmn + T\left(\frac{m}{2}, h\right) + T\left(\frac{m}{2}, n - h\right).$$

Expanding the recurrence (with ℓ and p the split points of the subproblems):

$$\begin{aligned} T(m, n) &= cmn + \left(c\frac{m}{2}h + T\left(\frac{m}{4}, \ell\right) + T\left(\frac{m}{4}, h - \ell\right)\right) + \left(c\frac{m}{2}(n - h) + T\left(\frac{m}{4}, p\right) + T\left(\frac{m}{4}, n - h - p\right)\right) \\ &= cmn + c\frac{m}{2}n + \sum T(\text{stuff}). \end{aligned}$$

How did we get rid of h ? **Answer:** linearity: $c\frac{m}{2}h + c\frac{m}{2}(n - h) = c\frac{m}{2}n$. Continuing the expansion,

$$T(m, n) = cmn + c\frac{m}{2}n + c\frac{m}{4}n + T(\text{stuff}) \cdots \leq 2cmn$$

(a non-rigorous jump, justified by the induction below).

Claim 1. $T(m, n) \leq 2cmn$.

Proof (induction on m). The base case is handled by choosing c large enough. For the inductive step, apply the hypothesis to the two subproblems of height $m/2$:

$$\begin{aligned} T(m, n) &= cmn + T\left(\frac{m}{2}, h\right) + T\left(\frac{m}{2}, n - h\right) \\ &\leq cmn + 2c\frac{m}{2}h + 2c\frac{m}{2}(n - h) \\ &= cmn + cmn = 2cmn. \end{aligned}$$

□

So the divide-and-conquer algorithm runs in $O(mn)$ time, only a factor of 2 more than the plain DP.

2.3.2 Space

Let $S(m, n)$ be the space to compute the path from $A[1..m]$ to $B[1..n]$. Then (plus the base case)

$$S(m, n) = \max\left(c(m + n), 1 + S\left(\frac{m}{2}, h\right), \frac{m}{2} + h + S\left(\frac{m}{2}, n - h\right)\right).$$

The terms come from:

- the work space $c(m + n)$ for the rolling-row DP at the current level;
- the two recursive calls, which are made *one after the other*, so their work space is reused; the second term accounts for what must be kept (e.g. the part of the output already written) while the other call runs.

The output is written into global variables (an array of size at most $m + n$, the length of the path). Together with the reusable work space $c(m + n)$, the total space is $O(m + n)$.

2.4 Summary

Task	Time	Space
Edit distance, standard DP	$O(mn)$	$O(mn)$
Edit distance, two rolling rows	$O(mn)$	$O(m + n)$
Edit sequence, DP with traceback table	$O(mn)$	$O(mn)$
Edit sequence, divide-and-conquer (today)	$O(mn)$	$O(m + n)$